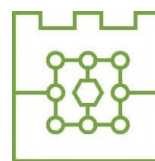




Politechnika Krakowska
im. Tadeusza Kościuszki
Wydział Informatyki i Telekomunikacji



Michał Bąk

Numer albumu: 144753

Porównanie metod uczenia przez wzmacnianie w Ultimate Texas Hold'em z niepełną informacją: wpływ reprezentacji stanu i funkcji nagrody na jakość strategii

Comparison of reinforcement learning methods in Ultimate Texas Hold'em under imperfect information: the impact of state representation and reward function on strategy quality

Praca dyplomowa magisterska
na kierunku Informatyka

Praca wykonana pod kierunkiem:
mgr inż. Piotr Biskup

Kraków, 2026

SPIS TREŚCI

1	Projekt i implementacja środowiska Ultimate Texas Hold'em	5
1.1	Założenia projektowe środowiska	5
1.2	Architektura rozwiązania	6
1.3	Model kart i sterowanie talią	8
1.3.1	Reprezentacja karty	8
1.3.2	Standardowa talia	8
1.3.3	Abstrakcja źródła kart i talia testowa	9
1.4	Ocena układów pokerowych	9
1.4.1	Reprezentacja wartości ręki	10
1.4.2	Ocena układu pięciokartowego	10
1.4.3	Wybór najlepszej ręki	11
1.5	Stan wewnętrzny i obserwacja agenta	11
1.5.1	Pełny stan rozdania	12
1.5.2	Obserwacja agenta	13
1.5.3	Projekcja stanu na obserwację	13
1.6	Maszyna stanów i przestrzeń akcji	14
1.7	Showdown i rozliczanie zakładów	16
1.7.1	Przebieg showdownu	16
1.7.2	Kwalifikacja krupiera	16
1.7.3	Model rozliczenia zakładów	17
1.7.4	Rozliczenie zakładu Blind	18
1.7.5	Rozliczenie Ante, Play i spasowania	18
1.8	Funkcja nagrody i wynik epizodu	19
1.9	Interfejs silnika	19
1.9.1	Tworzenie silnika i rozpoczęcie epizodu	19
1.9.2	Wykonanie decyzji	20
1.9.3	Właściwości publiczne	20
1.10	Rejestrowanie przebiegu i walidacja środowiska	21
1.10.1	Rejestrowanie historii i jej rekordy	21
1.10.2	Poziomy testowania	22
1.11	Ograniczenia środowiska	22

1.12 Podsumowanie	23
-----------------------------	----

1. Projekt i implementacja środowiska Ultimate Texas Hold'em

W niniejszym rozdziale przedstawiam projekt oraz implementację środowiska Ultimate Texas Hold'em wykorzystywanego w dalszej części pracy do uczenia i oceny agentów. Wcześniej opisane zasady gry odwzorowałem w postaci silnika, który jest odpowiedzialny za zarządzanie całym przebiegiem rozdania, udostępnia on agentowi legalne akcje do wykonania oraz rozlicza wynik finansowy rozdania.

Implementację podzieliłem na niezależne moduły odpowiedzialne między innymi za reprezentację kart, ocenę układów, sterowanie przebiegiem rozdania, walidację akcji oraz rozliczanie zakładów. Kodowanie obserwacji, odwzorowanie akcji i konstrukcja nagrody są poza warstwą realizującą zasady gry, co umożliwi mi porównywanie różnych reprezentacji stanu i funkcji nagrody.

1.1. Założenia projektowe środowiska

Podstawową jednostką interakcji ze środowiskiem jest jedno kompletne rozdanie (epizod). Rozdanie rozpoczyna się od utworzenia talii i przekazania graczowi oraz krupierowi po dwóch kart własnych, a kończy się spasowaniem gracza albo automatycznym rozstrzygnięciem układów.

Agent może podjąć decyzje wyłącznie w trzech etapach: przed flopem, po odkryciu flopa oraz po odkryciu wszystkich pięciu kart wspólnych. W zależności od etapu może czekać, wykonać zakład Play o odpowiednim mnożniku albo spasować. Po wykonaniu zakładu Play przez gracza środowisko automatycznie odkrywa pozostałe karty, ocenia ręce gracza i krupiera i przechodzi do stanu końcowego.

Wartości zakładów wyraziłem w jednostkach względnych, tzn. jedna jednostka odpowiada wartości zakładu Ante. Zakłady Ante i Blind mają zatem wartość jednej jednostki, natomiast wartość zakładu Play zależy od momentu jego wykonania i może wynosić od jednej do czterech jednostek. Zdecydowałem się podejść do tego w taki sposób, aby uniezależnić środowisko od konkretnej waluty i wysokości stawki.

Silnik zwraca szczegółowe rozliczenie poszczególnych zakładów oraz łączny wynik netto rozdania, będący podstawą nagrody terminalnej. Rozdzieliłem mechanizm rozliczania gry od funkcji nagrody - uzasadnienie tej decyzji przedstawiam w podrozdziale 1.8.

Zadbałem również o powtarzalność eksperymentów, przez co talia kart może być tasowana przy użyciu ziarna generatora liczb pseudolosowych, dzięki temu mogłem odtworzyć całą sekwencję rozdań. W testach jednostkowych zastosowałem dodatkowo talię deterministyczną, która pozwala określić kolejność dobieranych kart i zweryfikować konkretne przypadki przebiegu oraz rozliczenia rozdania.

1.2. Architektura rozwiązania

Środowisko Ultimate Texas Hold'em zaprojektowałem jako zestaw niezależnych modułów domenowych. Każdy moduł odpowiada za jeden określony obszar, taki jak reprezentacja kart, ocena układów, sterowanie przebiegiem rozdania albo rozliczanie zakładów. Centralnym elementem rozwiązania jest klasa `UTHGame`, która odpowiada za koordynowanie działania pozostałych komponentów, ale nie implementuje bezpośrednio wszystkich reguł gry.

Przyjąłem taki podział, aby ograniczyć zależności pomiędzy poszczególnymi częściami systemu. Model kart i ewaluacja układów są dzięki temu testowane niezależnie od zasad Ultimate Texas Hold'em. Analogicznie mechanizm rozliczania zakładów nie odpowiada za odkrywanie kart ani za walidację dozwolonych akcji. Dzięki temu zmiana jednego elementu, na przykład reprezentacji obserwacji agenta, nie wymaga modyfikowania sposobu oceny układów lub obliczania wypłat.

Najniższą warstwę rozwiązania stanowią moduły `cards` i `hands`. Pierwszy z nich definiuje karty oraz talię, natomiast drugi odpowiada za ocenę i porównywanie układów pokerowych.

Moduł `uth` zawiera modele stanu, reguły legalności akcji, mechanizm rozdawania kart, sposób rozliczania zakładów oraz logikę sterującą kolejnymi fazami rozdania. Klasa `UTHGame` pełni rolę menadżera tej warstwy. Na podstawie aktualnego stanu i wybranej akcji wywołuje ona odpowiednie operacje, aktualizuje stan rozdania oraz zwraca rezultat danego kroku.

Agent nie komunikuje się bezpośrednio z talią, ewaluatorem ani pełnym stanem wewnętrznym gry. Otrzymuje obiekt `UTHObservation`, a swoją decyzję przekazuje do metody `step`. Wynikiem wykonania akcji jest obiekt `StepResult`, zawierający kolejną obserwację oraz, w przypadku zakończenia rozdania, jego wynik i szczegółowe rozliczenie. Takie podejście pozwala mi rozgraniczyć logikę środowiska od implementacji agenta.

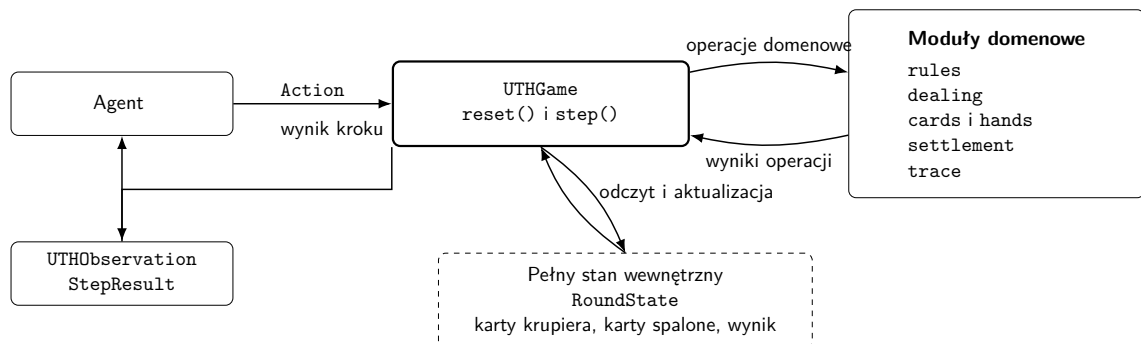
W tabeli 1.1 przedstawiłem najważniejsze moduły rozwiązania i zakres ich odpowiedzialności.

Co istotne silnik domenowy nie zależy od konkretnej biblioteki uczenia przez wzmocnianie. Integrację z takimi bibliotekami wydzieliłem do osobnej warstwy korzystającej z publicznego interfejsu `UTHGame` i przekształcającej obiekty domenowe na reprezentacje numeryczne. Pozwala mi to zachować jedną implementację zasad gry dla wszystkich badanych reprezentacji obserwacji, funkcji nagrody i metod uczenia.

Ogólny przepływ informacji pomiędzy agentem, klasą sterującą i pozostałymi elementami silnika przedstawiłem na rysunku 1.1. Agent komunikuje się wyłącznie z publicznym interfejsem klasy `UTHGame`. Pełny stan rozdania oraz moduły realizujące reguły gry pozostają ukryte za tą warstwą.

Tabela 1.1. Podział odpowiedzialności pomiędzy modułami środowiska UTH

Moduł	Odpowiedzialność
cards	Reprezentacja pojedynczej karty, jej rangi, koloru oraz talii składającej się z 52 unikalnych kart.
hands	Ocena pięciokartowych układów, wybór najlepszego układu z pięciu, sześciu lub siedmiu kart oraz porównywanie wartości rąk.
uth.models	Definicja pełnego stanu rozdania, obserwacji agenta oraz wyniku pojedynczego kroku środowiska.
uth.rules	Określanie zbioru legalnych akcji dla każdej fazy rozdania.
uth.dealing	Realizacja kolejności rozdawania kart, odkrywania flopa, turna i rivera oraz obsługa kart spalonych.
uth.card_source	Abstrakcja źródła kart oraz deterministyczna talia wykorzystywana w testach.
uth.settlement	Określanie wyniku rozdania, kwalifikacji krupiera oraz rozliczanie zakładów Ante, Blind i Play.
uth.game	Koordinacja przebiegu rozdania, walidacja akcji, przechodzenie pomiędzy fazami oraz tworzenie wyników kolejnych kroków.
uth.trace	Opcjonalne rejestrowanie obserwacji, decyzji agenta i końcowego stanu rozdania na potrzeby diagnostyki.



Rys. 1.1. Architektura i przepływ informacji w środowisku UTH

1.3. Model kart i sterowanie talią

Reprezentacja kart i mechanizm sterowania talią stanowią podstawową warstwę środowiska. Elementy te zaprojektowałem niezależnie od zasad Ultimate Texas Hold'em, dzięki czemu mogą być wykorzystywane także przez inne warianty pokera, o czym wspominałem w rozdziale poświęconym możliwym drogach rozwoju.

1.3.1. Reprezentacja karty

Pojedynczą kartę zaimplementowałem jako niemodyfikowalny obiekt `Card`, zawierający dwa pola: rangę oraz kolor. Rangi zakodowałem za pomocą typu wyliczeniowego `Rank`, natomiast kolory za pomocą typu `Suit`. Przez zastosowanie typów wyliczeniowych ograniczyłem możliwość utworzenia karty zawierającej niepoprawną wartość, a przez podejście do karty jako niemutowalnego obiektu zapewniłem bezpieczne umieszczanie jej w krotkach, zbiorach a także, przy porównywaniu liczebności zbioru czy wykrywaniu zduplikowanych kart.

Rangi kart przyjmują wartości liczbowe od 2 do 14. Karty od dwójki do dziesiątki zachowują swoje naturalne wartości, natomiast walet, dama, król i as otrzymują odpowiednio wartości 11, 12, 13 i 14. Zdecydowałem się na taką konwencję ponieważ umożliwia mi to bezpośrednie porównywanie rang podczas oceny układów.

Warto dodać, że as jest kartą najwyższą, natomiast szczególny przypadek strita od asa do piątki obsługiwany jest przez evaluator układów. Zbiór kolorów obejmuje trefle, karo, kiery i piki.

1.3.2. Standardowa talia

Klasa `Deck` reprezentuje standardową talię. Podczas jej tworzenia generowane są wszystkie kombinacje trzynastu rang i czterech kolorów. Liczba kart w pełnej talii wynosi zatem

$$|\mathcal{D}| = 13 \cdot 4 = 52. \quad (1.1)$$

Ponieważ każda kombinacja rangi i koloru występuje dokładnie raz, wygenerowana talia zawiera 52 unikalne karty.

Podstawową operacją jest metoda `draw()`, która pobiera jedną kartę z wierzchu talii i następująco ją usuwa z kolekcji. Jest również operacja służąca do pobrania wielu kart `draw_many()`. Próba pobrania karty z pustej talii lub większej liczby kart niż liczba dostępnych kart pozostałych w talii powoduje zgłoszenie wyjątku. Chciałem, aby takie zachowanie umożliwiała szybkie wykrycie niepoprawnego przebiegu rozdania.

Domyślnie talia jest tasowana bezpośrednio po utworzeniu. Zadbalem o możliwość utworzenia talii w konkretnej kolejności początkowej, co bywa użyteczne podczas testowania samego modelu kart.

Aby zapewnić powtarzalność eksperymentów, w klasie `Deck` zastosowałem możliwość przekazania ziarna generatora liczb pseudolosowych. Utworzenie dwóch talii z tym samym ziarnem prowadzi do uzyskania tej samej kolejności kart. Na poziomie klasy `UTHGame` zastosowałem dodatkowo generator nadrzędny, który przy każdym rozpoczęciu nowego rozdania generuje osobne ziarno przekazywane do nowej talii. Dzięki temu kolejne rozdania w obrębie jednego eksperymentu różnią się od siebie, ale cała ich sekwencja może zostać odtworzona przez ponowne uruchomienie środowiska z tym samym ziarnem początkowym. Jeżeli dwa eksperymenty zostaną uruchomione z tym samym ziarnem środowiska i zachowają tę samą kolejność wywołań, otrzymają identyczne talie, co ogranicza wpływ losowego doboru kart na ocenę różnic pomiędzy badanymi strategiami.

1.3.3. Abstrakcja źródła kart i talia testowa

Silnik rozdania nie zależy bezpośrednio od klasy `Deck`. Zamiast tego korzysta z abstrakcji `CardSource`, która wymaga jedynie udostępnienia operacji `draw()`. Każdy obiekt spełniający ten kontrakt może zostać użyty jako źródło kolejnych kart, dzięki czemu moduł odpowiedzialny za rozdawanie nie musi rozpoznawać, czy korzysta z przetasowanej talii, czy ze źródła deterministycznego.

Dzięki deterministycznemu źródłu byłem w stanie przygotować powtarzalne testy obejmujące określone scenariusze rozgrywki, kwalifikacji krupiera i rozliczenia zakładów. Do testowania konkretnych scenariuszy wprowadziłem klasę `FixedDeck`, która jest jawnie przekazaną sekwencją kart. Przed zapisaniem sekwencji sprawdzane jest, czy wszystkie jej elementy są poprawnymi obiektami `Card` oraz czy każda karta jest unikalna, dzięki czemu błąd w danych testowych nie może doprowadzić do rozdania zawierającego dwie identyczne karty.

1.4. Ocena układów pokerowych

Zadaniem modułu oceny układów pokerowych jest określenie wartości dowolnego poprawnego układu pięciokartowego oraz wybranie najlepszej możliwej ręki z pięciu, sześciu lub siedmiu dostępnych kart. W finalnej fazie podczas showdownu ewaluator jest wywoływany dla gracza i krupiera zawsze na siedmiu kartach, ponieważ w UTH karty wspólne są w całości odkrywane jeszcze przed showdownem, nawet gdy gracz postawi zakład `Play` najwcześniej możliwym momencie. Warianty pięcio- i sześciokartowe wykorzystuje natomiast agent regułowy opisany w rozdziale poświęconym agentom bazowym: ocenę pięciu kart (dwóch własnych i trzech kart flopa) przy podejmowaniu decyzji po flopie oraz ocenę sześciu kart (pięciu kart wspólnych i jednej hipotetycznej karty krupiera) przy liczeniu tzw. `outów` krupiera na riverze.

1.4.1. Reprezentacja wartości ręki

Do reprezentacji kategorii układów pokerowych podszedłem implementując typ wyliczeniowy `HandRank`. Kolejnym kategoriom przypisuje rosnące wartości liczbowe od 0 do 8, począwszy od wysokiej karty, a kończąc na pokerze. Dzięki temu porównanie kategorii odbywa się wprost przez porównanie odpowiadających im wartości całkowitych.

Sama kategoria nie wystarcza jednak do rozstrzygnięcia wszystkich przypadków (np. gracz i krupier mogą mieć układy tej samej kategorii, lecz różniące się rangą kart tworzących dany układ albo kartami dodatkowymi). Z tego względu pełna wartość ręki zaimplementowałem jako obiekt `HandValue`, zawierający kategorię układu `rank` oraz uporządkowaną krotkę wartości rozstrzygających remis `tiebreak`.

Klucz porównawczy ręki można zapisać jako

$$K(h) = (r(h), t_1(h), t_2(h), \dots, t_n(h)), \quad (1.2)$$

gdzie $r(h)$ oznacza wartość kategorii układu, natomiast $t_1(h), \dots, t_n(h)$ są kolejnymi wartościami rozstrzygającymi remis. Klucze porównywane są leksykograficznie. W pierwszej kolejności porównywana jest kategoria układu, a dopiero w przypadku jej równości - kolejne elementy krotki `tiebreak`.

Dla lepszego zobrazowania, wartość pary asów z królem, dziesiątką i dziewiątką jako kartami dodatkowymi może zostać zapisana w postaci

$$K(h) = (\text{ONE_PAIR}, 14, 13, 10, 9). \quad (1.3)$$

Jeżeli druga ręka również zawiera parę asów, porównanie przechodzi kolejno do króla, dziesiątki i dziewiątki. Kolory kart nie uczestniczą w rozstrzyganiu remisów, zgodnie ze standardowymi zasadami pokera.

Zadbałem aby obiekt `HandValue` sprawdzał zgodność długości krotki `tiebreak` z kategorią układu, walidował również, czy wszystkie jej elementy są liczbami całkowitymi odpowiadającymi poprawnym rangom kart. W ten sposób ograniczyłem możliwość utworzenia wartości ręki, która nie mogłaby powstać w prawidłowym rozdaniu.

1.4.2. Ocena układu pięciokartowego

Funkcja `evaluate_five_card_hand()` przyjmuje pięć unikalnych kart. W pierwszym kroku funkcja oblicza liczbę wystąpień każdej rangi. Na tej podstawie możliwe jest wykrycie par, dwóch par, trójki, fulla oraz karety. Niezależnie sprawdzane są dwa warunki: czy wszystkie karty mają ten sam kolor oraz czy pięć różnych rang tworzy ciąg kolejnych wartości.

Kategorie sprawdzane są od najsilniejszej do najsłabszej. Jeżeli spełnione są jednocześnie warunki strita i koloru, zwracany jest poker. W przeciwnym razie funkcja sprawdza kolejno kareta, full, kolor, strit, trójka, dwie pary, jedna para oraz wysoka karta.

Wartość `tiebreak` jest budowana bezpośrednio podczas rozpoznawania kategorii. Wynikiem funkcji nie jest jedynie nazwa układu, lecz kompletna wartość umożliwiającą porównanie z dowolną inną poprawną ręką. Szczególnym przypadkiem jest strit *A-2-3-4-5*, w którym `asa` pełni rolę najniższej karty, ewaluator zwraca wtedy strita o najwyższej karcie równej 5, bez zmiany wartości `asa` w pozostałych kategoriach.

1.4.3. Wybór najlepszej ręki

Podczas showdownu gracz i krupier dysponują łącznie siedmioma kartami. Rękę pokerową tworzy dokładnie pięć kart, dlatego funkcja `evaluate_best_hand()` generuje wszystkie możliwe pięciokartowe podzbiory dostępnych kart, ocenia każdy z nich, a następnie wybiera układ o największej wartości `HandValue`. Generowanie każdego możliwego podzbioru nie jest optymalnym rozwiązaniem, jednak liczba kombinacji jest niewielka i stała, co potraktowałem jako kompromis pomiędzy prostotą implementacji a wydajnością.

Liczba sprawdzanych kombinacji dla n dostępnych kart wynosi

$$N(n) = \binom{n}{5}. \quad (1.4)$$

Dla liczby kart obsługiwanych przez ewaluator otrzymuje się:

$$\binom{5}{5} = 1, \quad \binom{6}{5} = 6, \quad \binom{7}{5} = 21. \quad (1.5)$$

W przypadku standardowego showdownu UTH konieczne jest zatem sprawdzenie 21 kombinacji dla gracza oraz 21 kombinacji dla krupiera.

Przed wygenerowaniem kombinacji sprawdzane jest, czy przekazano od pięciu do siedmiu kart, czy wszystkie elementy są kartami oraz czy żadna karta nie występuje więcej niż raz.

Wynikiem operacji jest obiekt `EvaluatedHand`, który przechowuje zarówno wartość `HandValue`, jak i dokładnie pięć kart tworzących wybrany układ. Uznałem to za wygodny sposób prezentacji wyniku showdownu oraz diagnostyki ewaluatora.

Dla najwyższej kategorii, jaką jest poker królewski, nie tworzyłem osobnej kategorii `HandRank`. Zaprezentowałem go jako poker z asem jako najwyższą kartą.

1.5. Stan wewnętrzny i obserwacja agenta

Ponieważ Ultimate Texas Hold'em jest grą z niepełną informacją, odwzorowanie tej właściwości wymagało od mnie rozdzielenia pełnego stanu środowiska od informacji udostępnianych agentowi.

W implementacji zastosowałem dwa odrębne modele danych: `RoundState` przedstawiający pełny stan wewnętrzny rozdania oraz `UTHObservation` reprezentujący obserwację dostępną agentowi.

Agent nie powinien i nie otrzymuje odwołania do obiektu RoundState. Każda obserwacja jest tworzona na kanwie pełnego stanu przez osobną funkcję projekcji, która wyciąga ze stanu wyłącznie informacje dozwolone z perspektywy gracza.

1.5.1. Pełny stan rozdania

Obiekt RoundState zawiera wszystkie dane na temat rozdania. Zawiera on:

- aktualną fazę gry,
- dwie karty własne gracza,
- dwie ukryte karty krupiera,
- odkryte karty wspólne,
- karty spalone,
- mnożnik wykonanego zakładu Play,
- końcowy wynik rozdania,
- szczegółowe rozliczenie zakładów,
- rezultat showdownu.

Naturalnie nie wszystkie pola są aktywne jednocześnie. W stanach pośrednich wynik rozdania, rozliczenie, rezultat showdownu oraz mnożnik zakładu Play pozostają nieokreślone. Pola te otrzymują wartości dopiero po przejściu do stanu terminalnego.

Dla przykładu liczba kart wspólnych i spalonych jest zależna od aktualnej fazy. Ta zależność przedstawiam w tabeli 1.2.

Tabela 1.2. Liczba kart przechowywanych w stanie w poszczególnych fazach

Faza	Karty wspólne	Karty spalone	Znaczenie
PREFLOP	0	0	Rozdano wyłącznie karty własne gracza i krupiera.
FLOP	3	1	Spalono jedną kartę i odkryto flop.
RIVER	5	2	Odkryto komplet kart wspólnych.
TERMINAL	5	2	Rozdanie zostało zakończone fol-dem albo showdownem.

Obiekt stanu zaimplementowałem jako niemodyfikowalny, wymusza to utworzenie nowej instancji RoundState przy każdym przejściu środowiska (zamiast modyfikowania istniejącego obiektu). Zdecydowałem się na takie rozwiązanie, żeby ograniczyć ryzyko, że obiekt stanu może potencjalnie zostać niewłaściwie zmodyfikowany, ale też takie rozwiązanie ułatwia tworzenie deterministycznych testów.

Podczas tworzenia stanu sprawdzane są między innymi:

- poprawny typ fazy,
- obecność dokładnie dwóch kart gracza i dwóch kart krupiera,
- liczba kart wspólnych właściwa dla danej fazy,
- liczba kart spalonych właściwa dla danej fazy,
- brak zduplikowanych kart w całym stanie,
- zgodność pól końcowych z fazą rozdania.

Sprawdzone jest również, aby stan niekończący nie zawierał wyniku ani rozliczenia, stan zakończony foldem nie zawierał mnożnika Play ani rezultatu showdownu, natomiast stan zakończony showdownem powinien zawierać zarówno mnożnik wykonanego zakładu, jak i szczegóły ocenionych rąk.

1.5.2. Obserwacja agenta

Obiekt `UTHObservation` zawiera wyłącznie te dane, które mogą zostać wykorzystane przez agenta podczas podejmowania decyzji. Są to:

- aktualna faza gry,
- dwie karty własne gracza,
- dotychczas odkryte karty wspólne,
- zbiór legalnych akcji,
- mnożnik zakładu Play w obserwacji terminalnej.

W ramach tego obiektu umieściłem zbiór legalnych akcji, które agent może podjąć w danej fazie - dzięki temu zdejmuję z agenta konieczność odtwarzania zasad gry na podstawie fazy, co ułatwia implementację agentów bazowych.

Obserwacja `UTHObservation`, podobnie jak pełny stan, jest obiektem niemodyfikowalnym. Sprawdzana jest liczba widocznych kart, brak duplikatów oraz zgodność zbioru legalnych akcji z aktualną fazą gry.

1.5.3. Projekcja stanu na obserwację

Obiekt obserwacji dla agenta jest tworzony przez funkcję projekcji.

$$O_t = \phi(S_t), \quad (1.6)$$

gdzie S_t jest pełnym stanem środowiska w chwili t , natomiast O_t oznacza obserwację udostępnioną agentowi.

Dla warunków mojego środowiska funkcja projekcji ma postać

$$\phi(S_t) = (p_t, C_t^{player}, C_t^{community}, A_t^{legal}, m_t), \quad (1.7)$$

gdzie:

- p_t oznacza aktualną fazę gry,
- C_t^{player} oznacza karty własne gracza,
- $C_t^{community}$ oznacza odkryte karty wspólne,
- A_t^{legal} oznacza zbiór legalnych akcji,
- m_t oznacza mnożnik zakładu Play, jeżeli został już wykonany.

Karty krupiera i karty spalone należą do S_t , natomiast kolejność kart w talii jest przechowywana przez wewnętrzne źródło kart. Żadna z tych informacji nie jest elementem O_t , dokładny zakres pól obu obiektów przedstawiłem już w podrozdziale 1.5.1 i 1.5.2.

Ważne jest, że wynik rozdania, rozliczenie i szczegóły showdownu są po zakończeniu epizodu zwracane przez obiekt `StepResult`, a nie przez samą obserwację. Z racji, że te dane pojawiają się dopiero po wykonaniu ostatniej decyzji, nie mogą wpłynąć na wybór akcji w bieżącym rozdaniu.

1.6. Maszyna stanów i przestrzeń akcji

Przebieg pojedynczego rozdania Ultimate Texas Hold'em odwzorowałem jako maszynę stanów. Przejście do następnej fazy jest wyznaczone przez bieżącą fazę i akcję, z kolei zawartość stanu, tym samym obserwacji kolejnego stanu zależy od kart zwróconych przez źródło. Bieżąca faza określa zakres informacji widocznych dla agenta, zbiór legalnych akcji oraz operacje wykonywane przez środowisko po otrzymaniu decyzji.

Przebieg rozdania można przedstawić jako sekwencję faz:

$$\mathcal{P} = \{\text{PREFLOP}, \text{FLOP}, \text{RIVER}, \text{TERMINAL}\}. \quad (1.8)$$

Faza `TERMINAL` jest stanem absorbującym: nie ma dla niej żadnych legalnych akcji ($\mathcal{A}_{legal}(\text{TERMINAL}) = \emptyset$), a rozdanie wchodzi w nią wyłącznie w wyniku spasowania albo wykonania zakładu Play zakończonego showdownem - nie istnieje żadne inne przejście prowadzące do tej fazy.

Metoda `reset()`, wywoływana na początku każdego rozdania, rozdaje po dwie karty graczowi i krupierowi w kolejności P_1, D_1, P_2, D_2 , gdzie P to gracz a D to krupier, tworzy stan `PREFLOP` i zwraca pierwszą obserwację. Karty wspólne i spalone nie są jeszcze dobierane, to dzieje się w odpowiedniej fazie. Każde poprawne wykonanie metody `step()`

przesuwa rozdanie do kolejnej fazy albo do stanu terminalnego. Agent nie może pozostać w tej samej fazie po wykonaniu akcji.

Pełna przestrzeń akcji składa się z sześciu decyzji domenowych:

$$\mathcal{A} = \{\text{CHECK}, \text{BET_4X}, \text{BET_3X}, \text{BET_2X}, \text{BET_1X}, \text{FOLD}\}, \quad (1.9)$$

Maszyna stanów z podziałem na fazy i zbiory legalnych akcji wygląda następująco:

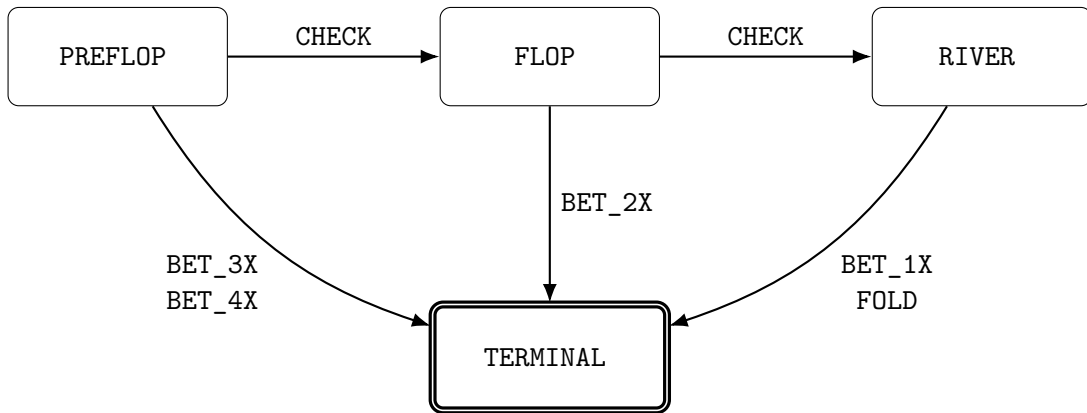
$$\mathcal{A}_{\text{legal}}(p) = \begin{cases} \{\text{CHECK}, \text{BET_3X}, \text{BET_4X}\}, & p = \text{PREFLOP}, \\ \{\text{CHECK}, \text{BET_2X}\}, & p = \text{FLOP}, \\ \{\text{BET_1X}, \text{FOLD}\}, & p = \text{RIVER}, \\ \emptyset, & p = \text{TERMINAL}. \end{cases} \quad (1.10)$$

Akcja CHECK odkłada decyzję o zakładzie Play i przesuwają rozdanie do następnej fazy, spalając kartę oraz odkrywając flop albo turn i river. Każda akcja typu BET ustala mnożnik Play wynikający z fazy: $4\times$ lub $3\times$ przed flopem, $2\times$ po flopie i $1\times$ na riverze.

Po zakładzie Play środowisko odkrywa pozostałe karty wspólne, przeprowadza showdown i przechodzi do stanu terminalnego. Akcja FOLD kończy rozdanie na riverze bez showdownu.

Struktura maszyny stanów gwarantuje, że zakład Play może zostać wykonany najwyżej raz w trakcie rozdania: każda akcja typu BET prowadzi bezpośrednio do stanu terminalnego, a akcja CHECK jedynie przesuwają decyzję do późniejszej fazy - nie istnieje przejście, które pozwoliłoby wykonać drugi zakład Play w tym samym rozdaniu.

Dla zilustrowania przebiegu rozdania posłużę się schematem przedstawionym na rysunku 1.2.



Rys. 1.2. Maszyna stanów pojedynczego rozdania UTH

1.7. Showdown i rozliczanie zakładów

Showdown jest fazą końcową każdego rozdania, w którym gracz wykonał zakład Play. Środowisko odkrywa wszystkie brakujące karty wspólne, ocenia najlepszą pięciokartową rękę gracza i krupiera, a następnie porównuje otrzymane wartości. W przypadku kiedy gracz spasa, nie dochodzi do showdownu.

1.7.1. Przebieg showdownu

Przypomnę, że podczas showdownu gracz i krupier dysponują siedmioma kartami. Ewaluator jest odpowiedzialny za wybór najlepszego pięciokartowego układu osobno dla gracza i krupiera. Wynik tej operacji jest przechowywany w obiekcie ShowdownResult, który zawiera:

- najlepszą rękę gracza,
- najlepszą rękę krupiera,
- wynik porównania z perspektywy gracza,
- informację o kwalifikacji krupiera.

Zbiór wszystkich możliwych wyników showdownu przedstawiam następująco:

$$\mathcal{O}_{showdown} = \{\text{PLAYER_WIN}, \text{DEALER_WIN}, \text{PUSH}\}. \quad (1.11)$$

Konkretny rezultat porównania rąk gracza i krupiera jest wyznaczany przez funkcję O : Dla wartości rąk H_p oznaczającej rękę gracza oraz H_d oznaczającej rękę krupiera, wynik showdownu można zdefiniować jako:

$$O(H_p, H_d) = \begin{cases} \text{PLAYER_WIN}, & H_p > H_d, \\ \text{DEALER_WIN}, & H_p < H_d, \\ \text{PUSH}, & H_p = H_d. \end{cases} \quad (1.12)$$

Wydawac by sie moglo, ze w momencie kiedy krupier sie nie kwalifikuje, nie ma sensu porównywać rąk obu stron. Rozważę jednak przykład sytuacji, gdzie każda ze stron ma jedynie wysokie karty (najniższy możliwy układ), ale najwyższa karta krupiera jest wyższa od najwyższej karty gracza. W takim przypadku krupier wygrywa rozdanie, mimo że się nie kwalifikuje. Dlatego porównuję rękę gracza i krupiera nawet w sytuacji, kiedy krupier się nie kwalifikuje.

1.7.2. Kwalifikacja krupiera

W Ultimate Texas Hold'em krupier kwalifikuje się, jeżeli posiada co najmniej jedną parę, taki warunek można zapisać w formie równania:

$$Q(H_d) = \begin{cases} 1, & \text{rank}(H_d) \geq \text{ONE_PAIR}, \\ 0, & \text{rank}(H_d) = \text{HIGH_CARD}. \end{cases} \quad (1.13)$$

gdzie $\text{rank}(H_d)$ oznacza rangę układu krupiera.

Samo zjawisko kwalifikacji krupiera nie zmienia wyniku porównania rąk, jednakże wpływa na sposób rozliczenia zakładu Ante, ponieważ:

- jeżeli krupier kwalifikuje się, zakład Ante wygrywa lub przegrywa zgodnie z wynikiem showdownu,
- jeżeli krupier nie kwalifikuje się, zakład Ante jest zwracany niezależnie od tego, która strona ma silniejszą rękę.

Odnośnie zakładu Play jest on rozliczany niezależnie od kwalifikacji krupiera w wyniku bezpośredniego porównania rąk. Z kolei zakład Blind jest rozliczany według układu gracza (w przypadku jego zwycięstwa), przy remisie zwracany, a przy przegranej gracza jest przegrywany.

Wszystkie wyżej opisane zależności w celu ułatwienia przedstawiam w tabeli 1.3.

Tabela 1.3. Rozliczenie zakładów po showdownie

Wynik	Kwalifikacja	Ante	Blind	Play
Wygrana gracza	tak	wygrana 1:1	według tabeli wypłat	wygrana 1:1
Wygrana gracza	nie	zwrot	według tabeli wypłat	wygrana 1:1
Remis	dowolna	zwrot	zwrot	zwrot
Wygrana krupiera	tak	przegrana	przegrana	przegrana
Wygrana krupiera	nie	zwrot	przegrana	przegrana

Kombinacja "Blind przegrany, Play zwrócony, Ante przepada" nie występuje w implementacji rozliczenia i nie znajduje się w tabeli 1.3 - Play jest zwracany wyłącznie przy remisie, a remis oznacza jednoczesny zwrot również Ante i Blind, nigdy ich utratę. Poza remisem Play zawsze wygrywa albo w całości przegrywa, więc powyższe pięć wierszy wyczerpuje wszystkie możliwe kombinacje.

1.7.3. Model rozliczenia zakładów

Zaprojektowałem obiekt `WagerSettlement`, który reprezentuje rozliczenie pojedynczego zakładu. Składa się on z `stake`, czyli początkowej wartości postawionego zakładu, oraz `net_profit`, czyli zysku albo straty netto.

Dla pojedynczego zakładu o stawce s i zysku netto n całkowity zwrot brutto wynosi $g = s + n$, jeśli $g = s$ oznacza to zwrot stawki, w przypadku $g = 0$ jej pełną utratę, a stawka $s = 0$ zakład niepostawiony.

Pełne rozliczenie rozdania reprezentuje obiekt `Settlement`, zawierający osobne wyniki dla Ante, Blind oraz Play. Łączny wynik netto S_{net} jest sumą zysków netto wszystkich trzech zakładów, można go zapisać w postaci równania:

$$S_{net} = n_{Ante} + n_{Blind} + n_{Play}. \quad (1.14)$$

Do reprezentacji wartości finansowych używam typu `Fraction`, ponieważ ten typ pozwala na przechowywanie wypłaty takiej jak $3/2$ (bez stosowania przybliżeń zmiennoprzecinkowych), w taki sposób unikam błędów zaokrągleń.

1.7.4. Rozliczenie zakładu Blind

Zakład Blind ma stałą stawkę równą jednej jednostce Ante. W sytuacji kiedy wygrywa gracz, zysk z racji tego zakładu jest uzależniony od tego jak wysoki układ uda się graczowi osiągnąć. Dla układów słabszych od strita Blind jest zwracany. Dla strita lub silniejszego układu stosowana jest tabela wypłat przedstawiona w tabeli 1.4.

Tabela 1.4. Tabela wypłat zakładu Blind

Układ gracza	Wypłata	Zysk netto
Wysoka karta	zwrot	0
Jedna para	zwrot	0
Dwie pary	zwrot	0
Trójka	zwrot	0
Strit	1:1	1
Kolor	3:2	$\frac{3}{2}$
Full	3:1	3
Kareta	10:1	10
Poker	50:1	50
Poker królewski	500:1	500

Zakład Blind jest stosowany wyłącznie wtedy, gdy gracz wygrał showdown. Przy remisie zakład jest zwracany, natomiast przy wygranej krupiera pełna stawka Blind zostaje utracona.

1.7.5. Rozliczenie Ante, Play i spasowania

Zakład Ante przy remisie jest zwracany. Jeżeli krupier kwalifikuje się, Ante wygrywa albo przegrywa zgodnie z wynikiem showdownu. W przypadku braku kwalifikacji zakład jest zwracany.

Zakład Play jest wypłacany 1:1, niezależnie od jego mnożnika. Żeby lepiej to zobrazować, posłużę się przykładem, powiedzmy że gracz postawił wygrany zakład Play $4\times$, daje zysk netto równy czterem jednostkom. Przy porażce tracona jest pełna wartość zakładu, natomiast przy remisie cała stawka zostaje zwrócona.

Jeżeli gracz spasuje na riverze, nie dochodzi do showdownu. Ante i Blind zostają przegrane, natomiast Play nie został postawiony.

Rozliczenie folda ma zatem postać

$$S_{fold} = (n_{Ante} = -1, n_{Blind} = -1, n_{Play} = 0). \quad (1.15)$$

Wtedy łączna stawka wynosi dwie jednostki, a wynik netto jest równy -2 .

W celu rozróżnienia zakładu niepostawionego od zakładu postawionego, ale zwróconego, w rozliczeniu Play zapisywana jest stawka równa zero.

1.8. Funkcja nagrody i wynik epizodu

Abym mógł sprawnie zmieniać warianty funkcji nagrody wykorzystywanej przez algorytmy uczenia przez wzmacnianie, w tym celu oddzieliłem mechanizm rozliczania zakładów od konstrukcji sygnału nagrody. Silnik domenowy wyznacza dokładny wynik finansowy rozdania zgodny z zasadami Ultimate Texas Hold'em. Natomiast konstruowanie sygnału nagrody na jego podstawie należy do warstwy integrującej silnik z agentem. Dzięki temu mogę w prosty sposób porównywać różne funkcje nagrody, bez ingerencji w zasady gry.

1.9. Interfejs silnika

Silnik domenowy zaimplementowałem w taki sposób, aby udostępniał niewielki interfejs publiczny, za pomocą którego można rozpocząć rozdanie, pobrać obserwację, wykonać decyzję oraz odczytać wynik zakończonego epizodu. Interfejs nie zależy od konkretnego algorytmu uczenia, co czyni go dość uniwersalnym rozwiązaniem do wykorzystania przy różnych eksperymentach.

Fundamentalnym elementem jest klasa `UTHGame`. Pojedyncza instancja tej klasy przechowuje stan aktualnego rozdania, źródło kart oraz opcjonalną historię decyzji. Po zakończeniu rozdania ta sama instancja może zostać wykorzystana ponownie przez wywołanie metody `reset()`.

1.9.1. Tworzenie silnika i rozpoczęcie epizodu

Konstruktor klasy `UTHGame` przyjmuje dwa opcjonalne parametry: `seed` oraz `record_history`. Ziarno określa powtarzalny strumień kart w talii generowanych dla kolejnych rozdawań, nie ustala jednak pojedynczej, niezmienniej talii dla całej instancji: ta sama instancja `UTHGame` obsługuje wiele rozdań pod rząd, ale każde wywołanie `reset()` tworzy dla niego nową talię, której ziarno jest pobierane z generatora śródowiska (mechanizm ten opisuję szerzej w podrozdziale 1.3.2). Rejestrowanie historii jest domyślnie wyłączone, służy ono do zbierania dodatkowych obiektów diagnostycznych.

Każde nowe rozdanie rozpoczyna się właśnie wywołaniem metody `reset()`, które zwraca pierwszą obserwację w fazie `PREFLOP`. W standardowym użyciu śródowisko samodzielnie tworzy przetasowaną talię. Metoda umożliwia również przekazanie własnego obiektu implementującego interfejs `CardSource`, na przykład deterministycznej talii `FixedDeck`.

Rozpoczęcie rozdania rozumiem jako wybór lub utworzenie źródła kart, rozdanie po dwie karty graczowi i krupierowi, utworzenie stanu PREFLOP, wyzerowanie historii bieżącego rozdania oraz utworzenie pierwszej obserwacji. Reset jest wykonywany transakcyjnie, silnik najpierw spróbuje pobrać wszystkie karty potrzebne do utworzenia stanu początkowego, a dopiero po powodzeniu zapisuje nowy stan i źródło kart. Jeżeli przekazane źródło nie może dostarczyć wymaganych kart, poprzedni stan silnika nie zostaje zastąpiony stanem częściowo utworzonego rozdania.

1.9.2. Wykonanie decyzji

Zaprojektowałem metodę `step()` tak, aby przyjmowała jedną wartość typu `Action`, wykonała jej odpowiadające przejście i zwracała obiekt `StepResult`, zawierający:

- obserwację po wykonaniu akcji,
- wynik rozdania (jeśli osiągnięto stan terminalny),
- rozliczenie zakładów w stanie terminalnym,
- szczegóły showdownu (jesli gracz nie spasował).

Wynik metody `step()` kroku pośredniego zawiera jedynie kolejną obserwację. Pola dotyczące wyniku i rozliczenia pozostają wtedy nieokreślone. W kroku terminalnym obiekt zawiera informacje niezbędne do obliczenia nagrody i zarejestrowania wyniku eksperymentu.

Właściwość `terminated` obiektu `StepResult` informuje, czy wykonana akcja zakończyła rozdanie. Jej wartość jest wyznaczana na podstawie fazy, która wchodzi w skład obserwacji.

Choć zbiór legalnych akcji jest wyznaczany przez środowisko i umieszczany w obserwacji, metoda `step()` nie ufa bezkrytycznie przekazanej decyzji - każda akcja jest walidowana ponownie przed jakąkolwiek zmianą stanu. W zależności od naruszonego warunku zgłaszany jest jeden z wyjątków: `RoundNotStartedError` (gdy nie wywołano jeszcze `reset()`), `RoundFinishedError` (gdy rozdanie jest już w fazie `TERMINAL`) albo `IllegalActionError` (gdy akcja nie należy do zbioru legalnego dla bieżącej fazy). Walidacja następuje przed dobieraniem kart, więc odrzucona decyzja nie zużywa kart z talii, nie zmienia fazy i nie jest dodawana do historii.

1.9.3. Właściwości publiczne

Poza metodami `reset()` i `step()` silnik udostępnia właściwości przedstawione w tabeli 1.5.

Właściwość `state` nie jest częścią interfejsu przeznaczonego do podejmowania decyzji przez agenta - udostępniłem ją jedynie do celów testowych i diagnostycznych.

Tabela 1.5. Publiczny interfejs klasy UTHGame

Element	Typ wyniku	Odpowiedzialność
reset()	UTHObservation	Rozpoczyna nowe rozdanie i zwraca obserwację preflop.
step(action)	StepResult	Wykonuje legalną akcję i zwraca wynik przejścia.
observation	UTHObservation	Zwraca aktualne informacje widoczne dla agenta.
state	RoundState	Zwraca pełny stan wewnętrzny, przeznaczony głównie do testów i diagnostyki.
is_started	bool	Informuje, czy rozpoczęto co najmniej jedno rozdanie.
history_enabled	bool	Informuje, czy rejestrowanie historii jest aktywne.
trace	RoundTrace lub None	Zwraca zapis bieżącego rozdania, jeżeli historia jest włączona.

Agent otrzymuje wartość właściwości `observation` albo obserwację zwróconą przez `reset()` lub `step()`.

1.10. Rejestrowanie przebiegu i walidacja środowiska

Bardzo ważnym elementem projektu środowiska jest pokrycie jego implementacji testami, ponieważ potencjalne niedopatrzenia na etapie implementacji, skrajne przypadki, błędy w logice bezpośrednio rzutują na wyniki eksperymentów, co może doprowadzić do nieefektywnego uczenia się agentów. Aby zapobiec takiemu scenariuszowi, dużą uwagę poświęciłem testom jednostkowym i integracyjnym, wprowadziłem opcjonalny mechanizm rejestrowania przebiegu rozdania oraz wzbogaciłem testy o weryfikację całych komponentów jak i ścieżek rozgrywki.

1.10.1. Rejestrowanie historii i jej rekordy

Rejestrowanie historii jest aktywowane parametrem `record_history` przekazywanym podczas tworzenia obiektu `UTHGame`. Domyślnie mechanizm ten pozostaje wyłączony, aby podczas uczenia nie generować dodatkowych obiektów diagnostycznych, które nie są potrzebne agentowi i zajmują pamięć oraz czas. Po jego włączeniu każde poprawnie rozpoczęte rozdanie otrzymuje dodatni, rosnący identyfikator `round_id`, a lista decyzji jest zerowana przy każdym poprawnym wywołaniu metody `reset()`. Właściwość `trace` zwraca obiekt `RoundTrace`; jeżeli historia jest wyłączona albo nie rozpoczęto jeszcze rozdania, jej wartością jest `None`. Mechanizm ten nie stanowi pamięci agenta i nie jest częścią obserwacji - wykorzystuję go do odtwarzania przebiegu wybranych rozdań w celu weryfikacji zgodności decyzji agenta z legalną przestrzenią akcji.

Niemodyfikowalny obiekt `DecisionRecord` przechowuje pojedynczą decyzję agenta, zawiera obserwację dostępną agentowi przed wykonaniem decyzji oraz wybraną akcję. Argumentem za zapisywaniem obserwacji przed przejściem jest to że, chciałem zachować

informacje dla jakiej obserwacji agent podjął konkretną decyzję, zapisywanie obserwacji po wykonaniu akcji byłoby w mojej ocenie niejednoznaczne, ponieważ taki stan zawierałby informacje już odkrytych kart, które mogłyby być skutkiem wykonanej akcji. Podczas tworzenia rekordu sprawdzane jest czy akcja należy do zbioru legalnego dla tej obserwacji.

Literalnie cały przebieg rozdania opisuje niemodyfikowalny obiekt `RoundTrace`, zawiera identyfikator rozdania, uporządkowaną krotkę rekordów decyzji oraz aktualny pełny stan `RoundState`. Obserwacje w rekordach zawierają wyłącznie informacje dostępne agentowi, natomiast końcowy stan jest pełnym obiektem diagnostycznym, z tego powodu `RoundTrace` nie jest przekazywany agentowi, lecz służy testom.

Zapis historii jest transakcyjny. Metoda `step()` zapamiętuje bieżącą obserwację, wykonuje walidację i przejście, a rekord dodaje dopiero po utworzeniu poprawnego kolejnego stanu. Jeżeli walidacja lub odkrywanie kart zakończy się wyjątkiem, rekord nie powstaje, więc historia nie zawiera akcji nielegalnych, decyzji po zakończeniu rozdania ani niedokończonych przejść. Podobny cel mi przyswiecał przy metodzie `reset()`, która zapisuje nowy stan dopiero po rozdaniu wszystkich czterech kart początkowych.

1.10.2. Poziomy testowania

Wspominałem, że testy są kluczowe, dlatego uporządkowałem je według odpowiedzialności komponentów. Testy jednostkowe sprawdzają w izolacji model kart i talie, ewaluator, modele stanu, rozdawanie, legalność akcji, showdown oraz rozliczenia. Testy integracyjne z kolei wykonują kompletne rozdania obejmujące wszystkie ścieżki maszyny stanów, od `reset()` do stanu terminalnego, wraz z rejestrowaniem historii.

Testy weryfikują również między innymi brak powtarzających się kart, zgodność liczby kart wspólnych i spalonych z fazą, brak danych terminalnych w stanie pośrednim, brak showdownu po spasowaniu, wykonanie co najwyżej jednego zakładu `Play`, brak kart krupiera i kart spalonych w obserwacji oraz brak zmiany stanu po odrzuconej akcji. Walidacja obejmuje także przypadki brzegowe i próby utworzenia stanów naruszających reguły domenowe.

W przypadku realizacji testów postawiłem na bibliotekę `pytest`. Podczas prac nad silnikiem analizowałem na bieżąco zarówno pokrycie instrukcji, jak i pokrycie rozgałęzień. Pozwoliło mi to wykryć nieprzetestowane ścieżki walidacji oraz przypadki brzegowe ewaluatora.

1.11. Ograniczenia środowiska

Choć uważam, że zaprojektowany i zaimplementowany przeze mnie model środowiska `Ultimate Texas Hold'em` jest wystarczający do przeprowadzenia badań nad strategiami podejmowania decyzji, to nie odwzorowuje on wszystkich aspektów gry w kasynie.

Prawdą jest, że zakłady `Trips` i `Progressive` nie wprowadzają dodatkowego momentu decyzyjnego, ponieważ ich wynik nie zależy od decyzji związanej z akcją `Play`. Nie

mniej zakłady poboczne wpływają na rozkład końcowych wypłat i utrudniałyby interpretację jakości strategii, a Progressive wymagałby dodatkowo modelu puli utrzymywanej pomiędzy rozdaniem, co naruszałoby niezależność epizodów. Oba zakłady poboczne można zaimplementować jako rozszerzenie warstwy rozliczeń co może stanowić przedmiot dalszych badań.

Silnik nie modeluje również salda gracza, sesji, ryzyka bankructwa ani zarządzania kapitałem, a wartości zakładów wyraża w jednostkach Ante, dzięki czemu strategię można porównywać niezależnie od nominału.

Zaprojektowałem środowisko, które reprezentuje pojedynczego gracza wobec krupiera i nie symuluje pozostałych miejsc przy stole, a zakryte karty innych graczy i tak byłyby nieznane, więc rozkład kart widocznych dla gracza odpowiadałby losowaniu bez zwracania ze standardowej talii.

Obserwacja jest obiektem domenowym zawierającym karty, fazę i zbiór legalnych akcji. W tej postaci jest czytelna i wygodna w testach, lecz nie może być bezpośrednio podana modelom uczenia maszynowego. Albowiem kodowanie `UTHObservation` do reprezentacji numerycznej pozostaje poza zakresem klasy `UTHGame`, ponieważ porównanie reprezentacji stanu jest jednym z elementów badawczych pracy. Wspólny silnik jest wygodny, bo pozwala zestawiać różne reprezentacje na identycznych rozdaniach i regułach.

1.12. Podsumowanie

W niniejszym rozdziale przedstawiłem projekt oraz implementację pojedynczego rozdania Ultimate Texas Hold'em. Silnik podzieliłem na niezależne moduły odpowiedzialne za reprezentację kart, ocenę układów, rozdawanie, przechowywanie stanu, walidację akcji, showdown oraz rozliczanie zakładów, a przebieg rozdania odwzorowałem jako maszynę stanów obejmującą fazy PREFLOP, FLOP, RIVER i TERMINAL, w której każda akcja BET kończy część decyzyjną i wymusza co najwyżej jeden zakład Play.

Ważnym elementem projektu jest rozdzielenie pełnego stanu `RoundState` od obserwacji `UTHObservation`, mianowicie agent otrzymuje wyłącznie własne karty, odkryte karty wspólne, fazę i legalne akcje, podczas gdy karty krupiera, karty spalone i kolejność talii pozostają wewnętrzne. Showdown korzysta z ogólnego ewaluatora wybierającego najlepsze pięć kart z siedmiu, a rozliczenia Ante, Blind i Play są przechowywane dokładnie za pomocą typu `Fraction`. Deterministyczne źródło `FixedDeck` oraz opcjonalny mechanizm historii zapewniają powtarzalność i możliwość analizy przebiegu rozdań.

Na tym etapie dostarczyłem silnik gry, który zapewnia podstawę do implementacji agentów bazowych, infrastruktury eksperymentalnej oraz metod uczenia przez wzmocnienie analizowanych w dalszej części pracy.

Spis rysunków

1.1	Architektura i przepływ informacji w środowisku UTH	7
1.2	Maszyna stanów pojedynczego rozdania UTH	15

Spis tabel

1.1	Podział odpowiedzialności pomiędzy modułami środowiska UTH	7
1.2	Liczba kart przechowywanych w stanie w poszczególnych fazach	12
1.3	Rozliczenie zakładów po showdownie	17
1.4	Tabela wypłat zakładu Blind	18
1.5	Publiczny interfejs klasy UTHGame	21